Closure says that free variables are closed over by a function that has them available via lexical scope, meaning that the function assigned to `getMostRecent` always has access to the `mostRecent` variable regardless of when or where it's invoked. As a result, even though the `makeMostRecent` function has already been invoked, the function returned from it still has access to its bound variables, via closure.

Closure and arguments

Function arguments are also closed over:

```
closedOverArgument = (x) ->
  -> x

five = closedOverArgument 5

nine = closedOverArgument 9

five()
# 5

nine()
# 9
```

In that sense, the arguments to a function act like locally defined variables. The scoping rules for arguments are the same as for variables with the exception that *argument names shadow* names in outer functions, whereas variable names don't.

3.6.3. Extended example: using closure

The new `watchedFileReader` function allows you to now solve the problem without having to have a global variable to share state around the program. Here's a modification of the program from listing 3.4 that uses closures and serves the combined contents of multiple files.

Listing 3.6. Serve multiple files